

Лабораторна робота 05

Локальне розгортання, навантаження і безпека

Курс «Прикладні ШІ-агенти та системи на основі великих мовних моделей» · 2026/2027

Лекції	9, 10, 18
Аудиторний час	3 год; самостійна робота 6 год
Формат	індивідуально
Вартість	0 грн: локальна модель, платні API не потрібні
Обладнання	<i>CPU-профіль</i> (гарантований): модель 0,5–1,5B, контекст 2k, конкурентність 1–4. <i>GPU-профіль</i> : 7–8B, контекст 8k, конкурентність до 8. Режим <i>stub</i> придатний лише для перевірки обгортки сервісу (крок 5), для вимірювань пунктів 1–4 і 6–9 потрібна жива модель

Мета

Розгорнути локальну мовну модель, виміряти її поведінку під навантаженням і показати експериментально, які захисти від *prompt injection* працюють, а які лише виглядають працюючими.

Теоретичні відомості

Генерація складається з двох фаз різної природи. Префіл обробляє весь запит одразу і впирається в обчислення; декод породжує токени по одному і впирається в пропускну здатність пам'яті, бо на кожен токен доводиться перерахувати ваги. Звідси й дві різні метрики: час до першого токена (TTFT) залежить від довжини запиту, а час на наступний токен (TPOT) — від пам'яті й розміру моделі. Повний час відповіді

$$T = \text{TTFT} + \text{TPOT} \cdot N_{\text{out}}$$

де N_{out} — кількість вихідних tokenів. Формула стандартна для інструментів вимірювання; якщо TPOT визначено строго як середній інтервал між токенами, точнішим є множник $N_{\text{out}} - 1$, бо перший токен уже врахований у TTFT. На довгих відповідях різниця несуттєва, на коротких — ні, тому в звіті вкажіть, яке означення ви використали.

Пам'ять споживають ваги, кеш ключів і значень та активації. Розмір KV-кешу

$$S_{KV} = 2 L H_{kv} d_h b s B$$

де L — кількість шарів, H_{kv} — кількість голів ключів і значень, d_h — розмірність голови, b — байтів на число, s — довжина послідовності, B — розмір пакета; двійка враховує окремо ключі й значення. Кеш зростає лінійно з довжиною контексту і кількістю одночасних запитів, тому саме він, а не ваги, зазвичай визначає межу конкурентності. Системи сервінгу керують ним сторінками, що усуває фрагментацію і підвищує пропускну здатність [1].

Квантизація зменшує кількість бітів на вагу і разом із нею — обсяг пам'яті та трафік до неї. Втрата якості залежить від моделі, схеми квантизації і задачі, тому її міряють на власній задачі, а не приймають на віру [2, 3].

Черга додає до затримки те, чого немає в самій моделі. За завантаження понад приблизно 80 відсотків час очікування зростає нелінійно, тому планування потужності починається з потрібної пропускну здатності в токенах за секунду, а не з кількості запитів. Кешем називають три різні речі: кеш префікса на боці сервера, кеш промпта на боці провайдера і семантичний

кеш на боці застосунку. Останній найризикованіший, бо близький запит не завжди означає той самий запит.

Безпекова частина починається з того, що недовірений текст стає інструкцією, а агент має права. Прямий `prompt injection` надходить у запиті користувача, непрямий — з документа, вебсторінки або відповіді інструмента, які потрапили в контекст [5]. Комбінація приватних даних, недовіреного контенту і каналу назовні описується як поєднання, за якого витік стає лише питанням часу. Виміряні захисти дають часткове зниження успішності атак, і адаптивні атаки частину з них знімають [6, 7]; жоден відомий захист не дає гарантій рівня криптографічних. Тому працює архітектура: розділення читання і запису, окремі облікові дані на інструмент, підтвердження людини на незворотних діях, ізоляція виконання і контроль вихідного трафіку [4, 8].

Результат вимірюють часткою успішних атак (`attack success rate`) до і після впровадження захисту на одному наборі сценаріїв, окремо для прямих і непрямих векторів.

Хід роботи

Позначка *ауд.* — крок виконується на занятті, *сам.* — самостійно в межах СРС. Довгі прогони (квантизації, сценарії атак, три рівні захисту) винесені на домашній етап. Аудиторна частина розрахована на 3 години.

1. **Розгортання (40 хв, ауд.).** Локальна модель під сервером. *Перевірка перед вимірюваннями:* довжина контексту і кількість паралельних запитів виставлені явно. Значення за замовчуванням тихо зіпсують усі подальші числа.
2. **Розрахунок пам'яті (40 хв, ауд.).** За формулами порахувати ваги і KV-кеш для двох довжин контексту, потім порівняти з фактом. *Перевірка:* розбіжність пояснена, а не списана на похибку.
3. **Затримки (50 хв, ауд.).** TTFT, TPOT і пропускна здатність при конкурентності 1, 2, 4, 8. *Перевірка:* перед кожним заміром — розігрів, інакше перший запит зіпсує р50.
4. **Дві квантизації (50 хв, сам.).** Порівняти за швидкістю і за якістю на 20 задачах з лабораторної 1 або 2. *Перевірка:* якість міряється на своїх задачах, а не переноситься з чужої таблиці.
5. **Обгортка сервісу (40 хв, сам.).** Бюджет часу, кеш, резервний варіант. *Перевірка:* показати спрацювання кожного механізму окремо.
6. **Сценарії атак (50 хв, сам.).** 10 прямих і 10 непрямих, захованих у документах. Для кожного наслідку заздалегідь записати *машинно перевірюваний* критерій успіху атаки — інакше підрахунок стане предметом суперечки. *Перевірка:* непрямі справді потрапляють у контекст через пошук, а не вставлені в запит руками.
7. **Базовий вимір (30 хв, ауд.).** Частка успішних атак *до* захисту, окремо для прямих і непрямих. Без цього числа наступний крок нічого не доводить.
8. **Три рівні захисту (60 хв, сам.).** Розмежування довіри в промпті, валідація виходу, архітектурне обмеження. Міряти *кожен окремо* на базовій конфігурації, а вже потім усі разом: увімкнені послідовно, вони змішують ефект захисту з порядком додавання.
9. **Пояснення результату (40 хв, сам.).** Які сценарії закрив промпт, які лише архітектура, а які не закрилися взагалі. Останні — з переліком залишкових контрзаходів. Двадцять сценаріїв — це пілот: одна атака важить 5 відсоткових пунктів, тому висновок формулюється як напрям, а не як виміряна ефективність.

Крок 9 і є результатом роботи. Список того, що *не* закрилося, цінніший за таблицю з нулями: він показує, що ви розумієте межі власної системи.

Завдання

1. Розгорнути локальну модель через `Ollama`, `llama.cpp` або `vLLM` — залежно від доступного обладнання — у двох квантизаціях (наприклад, 4- і 8-бітній). Зафіксувати конфігурацію обладнання і обраний профіль (CPU або GPU).
2. Для обраної моделі порахувати за формулами обсяг ваг і KV-кешу для двох довжин контексту і порівняти розрахунок із фактичним споживанням пам'яті. Пояснити розбіжність.
3. Виміряти TTFT, TPOT і пропускну здатність при конкурентності 1, 2, 4 і 8 (для CPU-профілю — 1, 2, 4) на однаковому наборі з 20 запитів. Побудувати таблицю затримок р50 і р95.

4. Порівняти дві квантизації за швидкістю і за якістю на 20 задачах з лабораторної 1 або 2. Сформулювати, чи виправдана втрата якості вигравшем у ресурсах.
5. Обгорнути модель сервісом із таймаут-бюджетом, кешем відповідей і резервним варіантом на випадок недоступності основної моделі. Показати спрацювання кожного механізму.
6. Скласти 20 сценаріїв *prompt injection*: 10 прямих у запиті користувача і 10 непрямих, захованих у документах, які потрапляють у контекст через пошук. Сценарії мають націлюватись на щонайменше три різні наслідки: розкриття системної інструкції, виклик забороненого інструмента, витік даних у відповідь. Для кожного наслідку записати машинно перевірюваний критерій успіху атаки (рядок-маркер, факт виклику інструмента, поява захищеного значення у відповіді).
7. Виміряти частку успішних атак на базовій конфігурації. Записати результат окремо для прямих і непрямих сценаріїв.
8. Впровадити три рівні захисту: розмежування довіреного і недовіреного тексту в промпті, валідація виходу, і архітектурне обмеження (запис лише з підтвердженням людини, окремі права на інструмент). Виміряти частку успішних атак для кожного захисту *окремо* відносно базової конфігурації і окремо для всіх трьох разом.
9. Пояснити результат: які сценарії закрилися промптом, які лише архітектурою, а які не закрилися взагалі. Для останніх запропонувати залишкові контрзаходи.
10. Здати *evidence package*: конфігурацію обладнання і моделі, таблиці затримок і якості, набір з 20 сценаріїв, таблицю успішності атак до і після кожного рівня захисту, схему меж довіри системи і висновок до 250 слів із переліком відомих обмежень вашого рішення. У висновку окремо зазначити, що вибірка з 20 сценаріїв є пілотною і не дає підстав для статистичних тверджень про ефективність захистів.

Контрольні запитання

1. Чому декод упирається в пропускну здатність пам'яті, а не в обчислення?
2. Як зміниться KV-кеш, якщо подвоїти довжину контексту, і як — якщо подвоїти пакет?
3. Чому саме кеш, а не ваги, зазвичай обмежує кількість одночасних запитів?
4. Що конкретно треба виміряти, щоб стверджувати, що квантизація «майже не погіршила якість»?
5. Чому p95 зростає нелінійно за високого завантаження?
6. Чим семантичний кеш небезпечніший за кеш префікса?
7. У чому різниця між прямим і непрямим *prompt injection* з погляду того, хто контролює текст?
8. Чому системний промпт не є механізмом безпеки?
9. Які сценарії з вашого набору неможливо закрити нічим, крім архітектури?
10. Що входить до *evidence package* і що саме кожен його елемент доводить?

Література

1. Kwon W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.
2. Lin J. et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys, 2024.
3. llama.cpp server, Ollama FAQ: контекст і паралелізм та vLLM engine arguments.
4. OWASP GenAI Security Project. OWASP Top 10 for LLM Applications, 2025.
5. Greshake K. et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISEC, 2023.
6. DeBenedetti E. et al. AgentDojo. NeurIPS, 2024.
7. DeBenedetti E. et al. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.
8. NIST AI 600-1. Artificial Intelligence Risk Management Framework: Generative AI Profile, 2024.
9. Xiao T., Zhu J. Foundations of Large Language Models. arXiv:2501.09223, 2025. Розділ 5.
10. Nasr M. et al. The Attacker Moves Second. arXiv:2510.09023, 2025.

Походження шаблону

Макет адаптовано з Laboratory report template for Basic Chemistry at Stockholm University, Viktor Bengtsson, LPPL 1.3c: article, Times-подібна гарнітура, широкі поля, fancyhdr, booktabs.